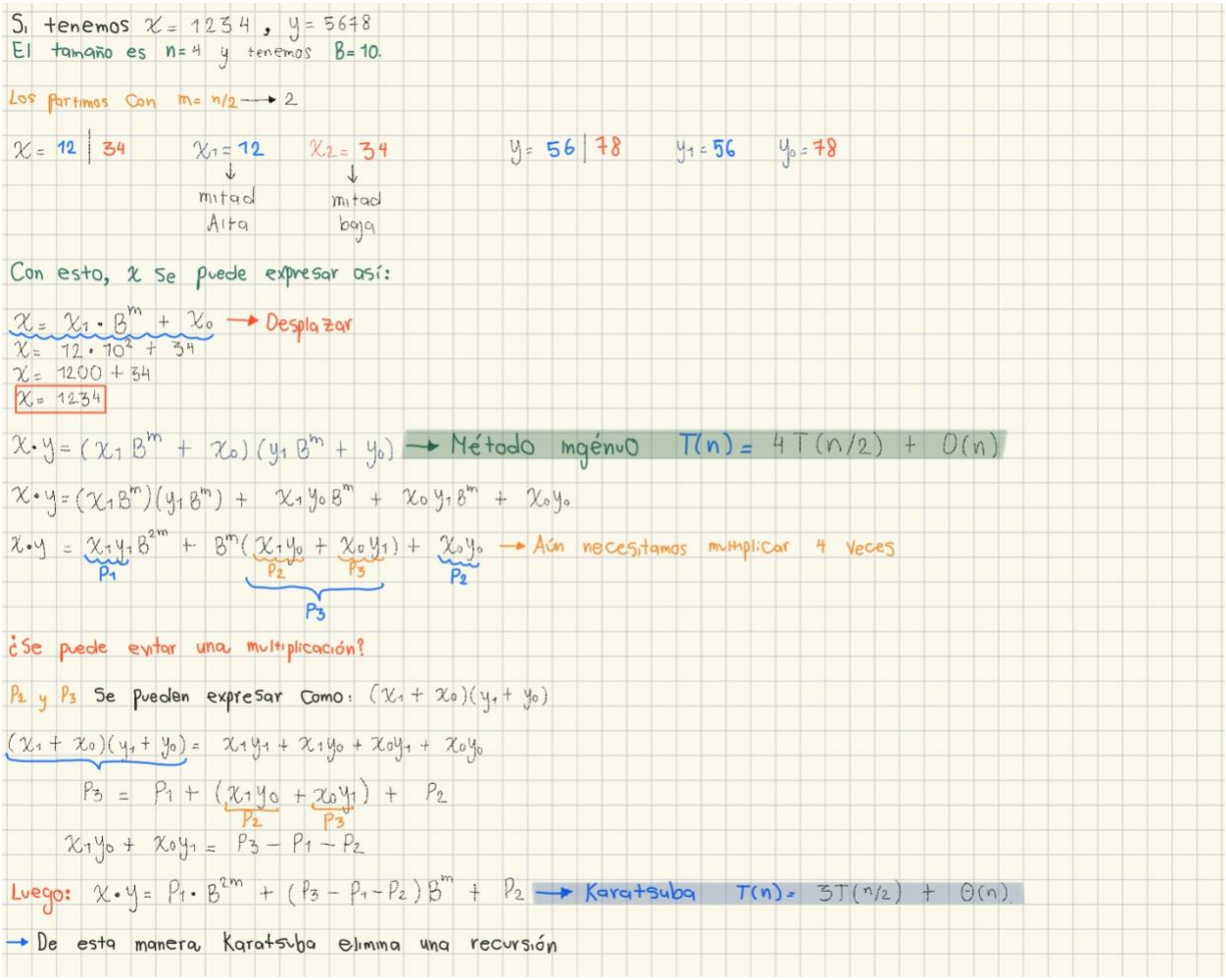
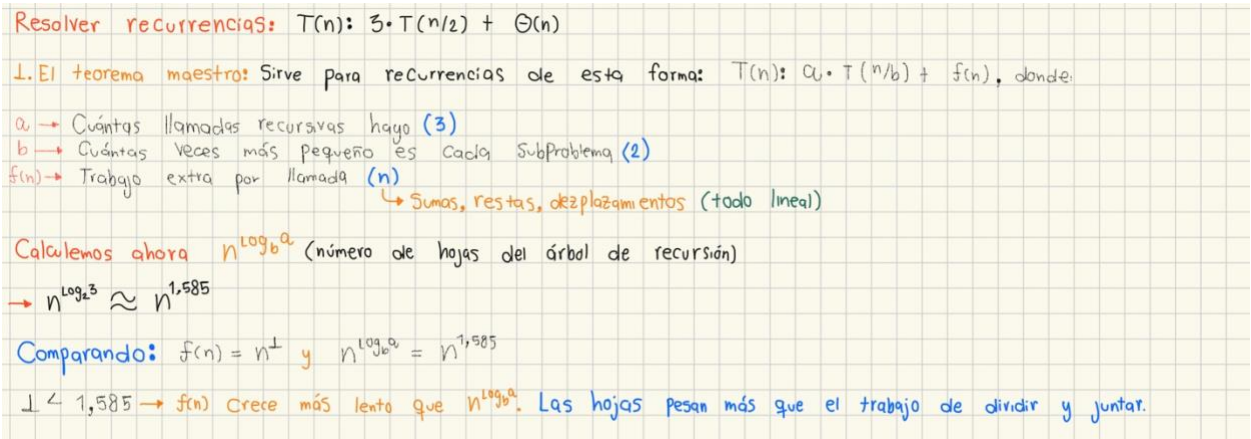
INFORME DE DESARROLLO

1. Deducción identidad de Karatsuba:

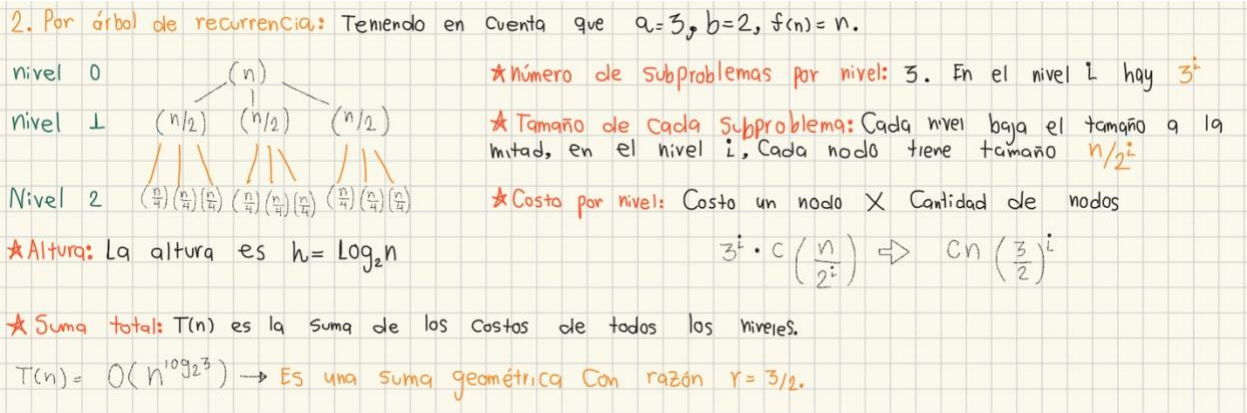


2. Resolver la recurrencia por ambos métodos:

2.1. Teorema maestro:



2.2. Árbol de recurrencia:



3. Análisis experimental

Montaje. Se midieron la **multiplicación clásica** y **Karatsuba** (versión pura, con caso base de un dígito) sobre los mismos operandos, compilando con g++ -std=c++17 -O2 y en el mismo equipo. Los tamaños fueron n = 2, 4, 8, …, 4096 dígitos; para cada uno se generaron cuatro pares de números aleatorios con semilla fija, y las repeticiones rotan entre ellos.

El tiempo se midió con **std::chrono::steady_clock**. Cada medición repite el cálculo hasta acumular al menos 0,1 s y cinco repeticiones y reporta el promedio por repetición, de modo que el tiempo supera con holgura la resolución del reloj; se hace un calentamiento previo y se conserva la mejor de tres pasadas para reducir el ruido. Cada resultado se consume con un **checksum** para que el compilador no elimine el cálculo, y se verificó que ambos algoritmos dieron el mismo producto en todos los tamaños.

Resultados. La Tabla 1 muestra el tiempo promedio por multiplicación y la razón entre el tiempo de la clásica y el de Karatsuba (una razón menor que 1 indica que la clásica es más rápida).

Tabla 1. Tiempo promedio por multiplicación según el tamaño de los operandos.

n (dígitos)	Clásica (µs)	Karatsuba (µs)	Clásica / Karatsuba
2	0.078	0.487	0.160
4	0.123	2.648	0.046
8	0.204	9.253	0.022
16	0.419	31.016	0.014
32	1.018	94.718	0.011
64	3.156	293.120	0.011
128	12.076	887.702	0.014
256	43.337	2 671.995	0.016
512	158.067	8 459.275	0.019
1024	615.619	24 478.712	0.025
2048	2 408.350	73 287.738	0.033
4096	9 649.542	223 204.688	0.043

La clásica fue más rápida en todos los tamaños: con n = 4096 tardó **9 649,5 µs** frente a **223 204,7 µs** de Karatsuba (unas 23 veces menos). Al ajustar una recta a log(tiempo) frente a log(n) para n ≥ 64, la pendiente fue de aproximadamente **1,92** para la clásica y **1,59** para Karatsuba.

Punto de cruce. Dentro del rango medido **no aparece punto de cruce**: la razón nunca supera 1 y en n = 4096 vale 0,043. La tendencia, no obstante, favorece a Karatsuba: al duplicar n, el tiempo de la clásica se multiplica por cerca de 4 y el de Karatsuba por cerca de 3, por lo que la razón crece alrededor de un 30 % por duplicación. Extrapolando, el cruce ocurriría en el orden de decenas de millones de dígitos (estimación, no medición).

Teoría frente a práctica. Las pendientes 1,92 y 1,59 coinciden con $\Theta(n^2)$ y $\Theta(n^{\log_2 3}) \approx \Theta(n^{1,585})$, lo que confirma la recurrencia $T(n) = 3T(n/2) + \Theta(n)$. Sin embargo, la complejidad

asintótica ignora las constantes: si el tiempo es $c_1 \cdot n^2$ para la clásica y $c_2 \cdot n^{1,585}$ para Karatsuba, el cruce está en $n^* = (c_2/c_1)^{(1/0,415)}$, que es muy grande cuando $c_2 \gg c_1$. En Karatsuba las **constantes ocultas** son altas: cada llamada recursiva copia y reserva memoria para las mitades, las sumas, las restas y los desplazamientos, y se llega hasta miles de llamadas sobre números de un dígito. La clásica, en cambio, recorre un arreglo contiguo con ciclos simples y sin asignaciones intermedias.

Números pequeños. Por eso, con pocos dígitos la clásica es más rápida: el costo fijo de Karatsuba pesa más que el ahorro de una multiplicación por nivel. La razón baja de 0,160 (n = 2) a un mínimo de 0,011 (n = 32 y 64) y desde ahí sube, es decir, hasta unos 64 dígitos domina la sobrecarga y luego empieza a notarse el menor crecimiento asintótico.

4. Conclusiones

Lo primero que nos quedó claro es que el truco de Karatsuba es sencillo, pero tiene mucho impacto. Si partimos los números a la mitad y multiplicamos de la forma obvia, necesitamos cuatro productos y la recurrencia $T(n) = 4T(n/2) + \Theta(n)$ nos deja otra vez en $\Theta(n^2)$: no ganamos nada. Al calcular $P_3 = (x_1+x_0)(y_1+y_0)$ y recuperar el término central como $P_3 - P_1 - P_2$, pasamos a solo tres subproblemas, $T(n) = 3T(n/2) + \Theta(n)$, y eso da $\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$. Cambiar una multiplicación por unas cuantas sumas y restas parece poca cosa, pero es justamente lo que baja el exponente.

Cuando medimos, nos alegró ver que los datos se comportaban como la teoría decía. Cada vez que duplicábamos el tamaño, la multiplicación clásica tardaba casi exactamente cuatro veces más (el exponente local llega a 1.96–2.00 entre 1024 y 4096 dígitos), mientras que Karatsuba tardaba alrededor de tres veces más, con exponentes entre 1.53 y 1.66 a partir de 32 dígitos, muy cerca de $\log_2 3 \approx 1.585$.

Lo que no vimos fue el punto de cruce. En todo el rango de 2 a 4096 dígitos la clásica fue más rápida: con 4096 dígitos tardó 9.6 ms, frente a 223 ms de Karatsuba, unas 23 veces menos. Eso sí, la distancia se fue acortando: la razón clásica/Karatsuba pasó de 0.011 en n = 32 a 0.043 en n = 4096. Si extrapolamos las dos tendencias (×4 contra ×3 por cada duplicación), el cruce aparecería alrededor de los 8 millones de dígitos, muy lejos de lo que medimos.

Esto nos enseñó que la complejidad asintótica no cuenta toda la historia. La notación Θ esconde constantes, y en nuestra implementación esas constantes son grandes. El caso base es de un solo dígito, así que para n = 4096 se hacen del orden de $3^{12} \approx 530\,000$ llamadas hoja, y cada una crea un vector dentro de multiplicacionClasica. Además, en cada nivel de la recursión se crean muchas cadenas temporales: partir usa substr, sumar y restar invierten y recortan cadenas, y los parámetros se pasan por valor. La clásica, en cambio, es un doble ciclo sobre un único arreglo que se reserva una sola vez. Entendimos por qué las bibliotecas reales combinan ambos métodos: usan Karatsuba para números grandes y cambian a la clásica por debajo de cierto umbral.

Por último, tener la multiplicación clásica como referencia nos dio mucha tranquilidad. Comparar Karatsuba contra ella en todos los casos de borde que pedía la guía, comprobar propiedades algebraicas y correr 5000 pares aleatorios nos permitieron confiar en el código: las 286 comprobaciones pasaron.

5. Dificultades encontradas durante el desarrollo

El principal desafío de esta entrega fue implementar operaciones matemáticas sobre cadenas (strings) y hacer las validaciones que esto implicaba.

Entender y posteriormente deducir la teoría de Karatsuba también fue un reto interesante que requirió tiempo y dedicación.

6. Uso de IA:

Se utilizó el modelo Sonet 5.5 de Claude para entender más a fondo el documento de la entrega y saber específicamente cuáles eran los requisitos y entregables. Adicionalmente, con el mismo se dividieron las tareas de los integrantes.

También se utilizó para la solución de dudas respecto a la compilación de las pruebas:

